SOFTWARE REQUIREMENT SPECIFICATION (SRS)

ON

Delhi Metro Smart Ticketing & Biometric Authentication System

BACHELOR OF TECHNOLOGY — Computer Science & Engineering

Name	GEU Roll No.	Class Roll No.	Role
Shashi Pandey	2023609	GEU-51	Frontend, Algorithm & Data Logic Developer
Mohd Kushaad	2023889	GEU-34	Frontend, UI/UX & Backend Developer
Himadri Mani	2418026	GEHU-15	Export & DevOps Developer
Yash Agrawal	2024201	GEU-65	Database, Architect & Biometric Developer

Submitted To: Dr. Neha Tripathi
Department of Computer Science & Engineering
Graphic Era (Deemed to be University) — February 2026

TABLE OF CONTENTS

Section	Title	Page
1	Introduction & Problem Definition	3
2	Overall System Description	4
3	System Modules	5
4	Functional Requirements	6
5	Non-Functional Requirements	7
6	System Architecture	8
7	Database Design (5 Tables from ER Diagram)	8
8	Software & Hardware Requirements	9
9	Constraints & Assumptions	10
10	References	10

1. Introduction & Problem Definition

1.1 Project Overview

The Delhi Metro is used by millions of people every day. Managing tickets for such a large crowd is a big challenge. Traditional systems that use tokens, smart cards, or QR codes are slow, require physical contact, and often create long queues at station gates. This project — the Delhi Metro Smart Ticketing and Biometric Authentication System — solves this problem by combining two existing projects: the Smart Metro Navigator (developed by Shashi Pandey) and the MFAS — Multi-Factor Authentication System (developed by Yash Agrawal).

The merged system introduces a 'Face as a Ticket' approach. This means a commuter can register once and, from that point on, simply walk up to a metro gate, let the camera scan their face, and the gate will open — no card, no token, no app needed at that moment. Behind the scenes, the system finds the best route using graph algorithms and automatically deducts the correct fare from the user's wallet.

1.2 Purpose of This Document

This Software Requirement Specification (SRS) document is written to:

- Clearly explain what the system is supposed to do.
- List all the features and functions that need to be built.
- Describe the technology that will be used.
- Give developers, testers, and the academic supervisor (Dr. Neha Tripathi) a complete reference document for building and evaluating the project.

1.3 Scope of the Product

The system will be a software application covering the following capabilities:

- Biometric enrollment: Users register their face and create a digital wallet.
- Smart route finding: Given a starting station and a destination, the system finds the best route using Dijkstra's or A* algorithm across all 285+ Delhi Metro stations.
- Face-gate authentication: At entry and exit gates, the system scans the face, verifies the user, and opens the gate.
- Automatic fare deduction: Based on the distance traveled, the correct amount is deducted from the user's wallet.
- Mobile/web interface: Users can check their balance, journey history, and top up their wallet.

1.4 Definitions, Acronyms & Abbreviations

Term	Full Form / Meaning
SRS	Software Requirement Specification
DMRC	Delhi Metro Rail Corporation
AI	Artificial Intelligence

CNN	Convolutional Neural Network — a type of AI model used for image/face recognition
MFAS	Multi-Factor Authentication System — the existing face recognition project by Yash Agrawal
API	Application Programming Interface — a way for two software systems to talk to each other
RDBMS	Relational Database Management System — e.g., PostgreSQL
LLM	Large Language Model — AI for conversational chatbot assistance
OTP	One-Time Password — used as a backup for authentication
DXA	Unit of measurement used in document design (1440 DXA = 1 inch)

1.5 References

- IEEE Standard 830-1998: Recommended Practice for Software Requirements Specifications.
- Shashi Pandey, Smart Metro Navigator — GitHub: github.com/Shashi0704/SMART_METRO_NAVIGATOR
- Yash Agrawal, MFAS — GitHub: github.com/IMYashAgrawal/MFAS
- DMRC Official Network Map and Fare Metrics, 2026.
- M. Malamut, 'Dijkstra's Pathfinder: Shortest Path Analysis in the Washington D.C. Metro,' Coastal Carolina University, 2021.

2. Overall System Description

2.1 What Problem Are We Solving?

Current metro ticketing systems have several issues:

- Long queues at token/card vending machines.
- Difficulty for elderly or first-time commuters who do not know the fare structure.
- No intelligent route suggestions that account for real-time interchange and line status.
- Physical contact with machines (hygiene concern, especially post-pandemic).

Our system removes all of these pain points by making a user's face their metro pass, and an AI assistant their personal route planner.

2.2 How the System Works — A Simple Walkthrough

Step 1: One-Time Registration

The user downloads the app or visits the kiosk. They provide their name, phone number, and a clear photo of their face. The system stores a mathematical representation of the face (called a 'face embedding') — not the actual photo — so it is privacy-safe. The user also sets up a digital wallet and can add money to it.

Step 2: Planning a Journey

The user opens the app and types or speaks their source station and destination station. The system's route engine instantly calculates the best path — showing the route, total time, number of interchanges, and the expected fare.

Step 3: Entering the Metro

The user walks to the entry gate. A camera scans their face. Within milliseconds, the system matches the face with the database, confirms the user's identity, checks that the wallet has enough balance, and opens the gate. The entry station and time are recorded.

Step 4: Exiting the Metro

The same process happens at the exit gate. The system records the exit station, calculates the fare (based on distance between entry and exit), and automatically deducts the amount from the wallet. A notification is sent to the user's phone.

Step 5: Wallet Top-Up & History

The user can view their journey history and top up the wallet anytime via UPI, debit card, or net banking through the app.

2.3 Product Perspective

This is a standalone, end-to-end system. It is not a plugin for an existing DMRC app — it is a complete new platform that can be piloted independently or eventually integrated with official DMRC infrastructure. The system interacts with:

- Hardware cameras at metro gates (for biometric capture).

- A central PostgreSQL database (for user profiles, biometrics, transactions).
- A mobile/web frontend (for users).
- A backend API layer (FastAPI/Node.js) that connects all components.

2.4 User Types

User Type	Who They Are	What They Can Do
Regular Commuter	Any person who travels on the Delhi Metro	Register face, plan routes, enter/exit gates, manage wallet
Admin / DMRC Staff	Station managers or IT staff	Manage user accounts, view logs, update fare tables, monitor system health
Guest / Visitor	A first-time or unregistered user	Use the route planner only (cannot use face-gate entry)

3. System Modules

The system is divided into five separate modules. Each module handles one specific part of the system, making it easier to build, test, and maintain.

Module No.	Module Name	What It Does
Module 1	Biometric Enrollment & User Management	Handles new user registration. Captures the face using a camera, converts it into a 128-dimensional digital vector (face embedding) using a CNN model, and saves it securely in the database. Also manages user profile updates and account deactivation.
Module 2	Intelligent Route Navigator	Takes a source and destination station as input. Uses Dijkstra's Algorithm (or A* for faster results) to calculate the shortest path through the DMRC network of 285+ stations. Returns route options: 'Shortest Time' and 'Minimum Interchanges'. This is built on top of the existing Smart Metro Navigator project.
Module 3	Face-Gate Authentication (Edge Inference)	Runs at the metro turnstile. Uses a live camera feed, detects a face in real time using OpenCV and Dlib, converts it to an embedding, and matches it against the database. If the match confidence is above the threshold and the wallet balance is sufficient, the gate opens. This is powered by the MFAS project.
Module 4	Financial Transaction & Wallet Engine	Manages each user's wallet balance. Calculates the fare based on the entry and exit stations using the DMRC official fare slab. Handles auto-debit on exit, top-up payments, and generates a detailed transaction log.
Module 5	AI Chatbot Assistant (Optional / Future)	A conversational assistant powered by an LLM that answers questions like 'Which station has a lift?' or 'What is the fare from Rajiv Chowk to Noida Sector 18?' This module is planned for a future version.

4. Functional Requirements

Functional requirements describe what the system must be able to do. Each requirement is written in plain language.

4.1 User Registration & Biometric Enrollment

- FR-01: The system must allow a new user to register by providing First_name, Last_name, Email, Phone_no, and a live face scan.
- FR-02: The system must extract a 128-dimensional face embedding from the scanned face and store it as Biometric_data in the Biometric Detail table. The raw photo must never be stored.
- FR-03: The system must automatically create a Wallet record for the new user with a Current_balance of 0.00 after successful registration.
- FR-04: Each user must have a unique User_id. The Registration_date must be auto-recorded at the time of signup.
- FR-05: Face recognition acts as Multi-Factor Authentication (MFA). The user does not need a password or OTP — their face IS their identity proof for login, gate entry, gate exit, and wallet top-up.

4.2 Route Planning

- FR-06: The user must be able to select a source station and a destination station.
- FR-07: The system must calculate and display the optimal route with station-by-station directions.
- FR-08: The system must display the expected travel time and number of interchanges.
- FR-09: The system must display the expected fare before the journey begins.
- FR-10: The system must offer at least two route options: Shortest Time and Minimum Interchanges.

4.3 Gate Entry & Exit (Face Recognition as MFA)

- FR-11: The gate camera must detect and capture a face within 2 seconds of the user standing in front of it.
- FR-12: The system must match the captured face against the Biometric_data stored in the Biometric Detail table and return a result within 1 second. No password or OTP is required.
- FR-13: If the face matches and the Current_balance in the Wallet table is sufficient, the gate must open. A new Transaction record must be created with the Source_station and User_id.
- FR-14: If the face does not match or the wallet balance is insufficient, the gate must remain closed and display an appropriate error message.
- FR-15: On successful exit, the system must update the Transaction record with the Destination_station and Fare_amount, and deduct the fare from Current_balance in the Wallet table.
- FR-16: Face recognition at the exit gate also acts as MFA — confirming the same user who entered is the one exiting before completing the fare deduction.

4.4 Fare Calculation & Wallet Management

- FR-17: The system must calculate the correct Fare_amount based on official DMRC fare slabs and the distance between Source_station and Destination_station stored in the Transaction table.
- FR-18: The Fare_amount must be automatically deducted from Current_balance in the Wallet table upon successful face-scan exit.
- FR-19: The system must allow users to top up their wallet via the app using UPI, card, or net banking. Face recognition (MFA) must verify the user's identity before the top-up is processed — no OTP needed.
- FR-20: The system must send a notification (in-app or SMS to Phone_no) after every fare deduction and top-up.
- FR-21: Users must be able to view their last 30 Transaction records showing Source_station, Destination_station, and Fare_amount.

4.5 Admin Functions

- FR-22: Admins must be able to view all registered users and their wallet balances.
- FR-23: Admins must be able to deactivate a user account.
- FR-24: Admins must be able to update fare tables.
- FR-25: Admins must have access to system-level logs for monitoring and troubleshooting.

5. Non-Functional Requirements

Non-functional requirements describe how the system must perform — speed, security, reliability, etc.

ID	Category	Requirement
NFR-01	Performance	The face recognition at the gate must complete within 1 second.
NFR-02	Performance	The route calculation must return results within 3 seconds for any source-destination pair.
NFR-03	Scalability	The database must support at least 100,000 registered users without any slowdown.
NFR-04	Security	Face embeddings must be encrypted at rest and in transit using AES-256.
NFR-05	Security	All API communication must use HTTPS with TLS 1.2 or higher.
NFR-06	Reliability	The system must have at least 99% uptime (allowing no more than ~87 hours downtime per year).
NFR-07	Accuracy	The face recognition model must achieve a minimum accuracy of 98% under standard lighting conditions.
NFR-08	Usability	A new user must be able to complete the registration process in under 3 minutes.
NFR-09	Privacy	Raw face images must never be stored. Only embeddings are stored, making the system GDPR-aligned.
NFR-10	Portability	The mobile app must run on both Android (8.0+) and iOS (12.0+).

6. System Architecture

6.1 Architecture Overview

The system follows a three-tier architecture:

- Presentation Layer (Frontend): The mobile app (React Native or Flutter) and web app that the user interacts with.
- Application Layer (Backend): FastAPI (Python) or Node.js server that handles business logic — routing, authentication, fare calculation.
- Data Layer (Database): PostgreSQL database that stores user profiles, face embeddings, transaction records, and station data.

At the metro gate, an Edge Device (a small computer like a Raspberry Pi or Jetson Nano) runs the face detection and matching locally to minimize latency. It communicates with the central backend over a secure API.

6.2 Technology Stack

Layer	Technology Used	Purpose
Frontend	React Native / Flutter	Mobile app for user registration, route planning, wallet management
Frontend	HTML5, CSS3, Bootstrap, JavaScript	Web interface for the same features
Backend	FastAPI (Python 3.9) / Node.js	API server that connects all modules
AI / CV	OpenCV, Dlib, TensorFlow	Face detection, embedding extraction, matching
Route Algorithm	Java (Dijkstra's / A*)	Path finding across 285+ stations (from Smart Metro Navigator)
Database	PostgreSQL 13+	Stores all user data, embeddings, transactions, station info
Edge Device	Python + OpenCV on Raspberry Pi / Jetson	Real-time face capture and matching at gate turnstiles
Version Control	Git / GitHub	Code management and collaboration

7. Database Design

The system uses a PostgreSQL relational database with 5 tables as defined in the ER Diagram. Each table is described below with its fields, data types, and constraints.

The ER Diagram shows the following relationships:

- One User can have ONE Biometric Detail record (one-to-one).
- One User can have ONE Wallet (one-to-one).
- One User can have MANY Transactions (one-to-many).
- One Station can appear in MANY Transactions as source or destination (one-to-many).

7.1 Table: User

Stores the personal details of every registered commuter.

Field Name	Data Type	Constraints	Description
User_id	SERIAL / INT	PRIMARY KEY, NOT NULL	Unique identifier for each user. Auto-generated.
First_name	VARCHAR(50)	NOT NULL	User's first name.
Last_name	VARCHAR(50)	NOT NULL	User's last name.
Email	VARCHAR(100)	UNIQUE	User's email address for notifications.
Phone_no	VARCHAR(15)	NOT NULL, UNIQUE	Registered mobile number for SMS alerts.
Registration_date	DATE / TIMESTAMP	DEFAULT NOW()	The date and time the user registered.

7.2 Table: Biometric Detail

Stores the face embedding (mathematical face vector) for each user. This is separate from the User table to keep biometric data isolated and secure. Face recognition is used as Multi-Factor Authentication (MFA) — it replaces the need for a password or OTP during both login and ticket purchase. When a user wants to log in or buy a ticket, the camera scans their face, and the system matches it against the stored Biometric_data. If matched, access is granted instantly — no password, no OTP required.

Field Name	Data Type	Constraints	Description
Biometric_id	SERIAL / INT	PRIMARY KEY, NOT NULL	Unique identifier for each biometric record.

User_id	INT	FOREIGN KEY (User.User_id), NOT NULL	Links this biometric record to its owner. One user = one record.
Biometric_data	BYTEA / FLOAT[]	NOT NULL	128-dimensional face embedding vector extracted by the CNN model. Raw photos are never stored — only this mathematical representation.
Last_update	TIMESTAMP	DEFAULT NOW()	Tracks when the biometric data was last updated (e.g., after re-enrollment).

7.3 Table: Station

Stores information about each Delhi Metro station. This data is used by the Route Navigator module to build the metro graph and calculate shortest paths.

Field Name	Data Type	Constraints	Description
Station_id	SERIAL / INT	PRIMARY KEY, NOT NULL	Unique identifier for each station.
Station_name	VARCHAR(100)	NOT NULL, UNIQUE	Official name of the station (e.g., Rajiv Chowk, Noida Sector 18).
Line_color	VARCHAR(50)	NOT NULL	The metro line this station belongs to (e.g., Blue Line, Yellow Line, Red Line).
Coordinates	VARCHAR(100) / POINT		GPS coordinates (latitude, longitude) of the station for map display.

7.4 Table: Wallet

Each user has exactly one wallet. The wallet holds the current balance that is used for automatic fare deduction when the user exits a metro station.

Field Name	Data Type	Constraints	Description
Wallet_id	SERIAL / INT	PRIMARY KEY, NOT NULL	Unique identifier for each wallet.
User_id	INT	FOREIGN KEY (User.User_id), NOT NULL, UNIQUE	Links the wallet to its owner. One user = one wallet only.
Current_balance	DECIMAL(10,2)	DEFAULT 0.00, CHECK >= 0	The current money available in the wallet (in INR). Cannot go below zero.

7.5 Table: Transaction

Records every journey taken by a user. When a user enters a station, a record is created. When they exit, the record is updated with the destination station and the fare is deducted from the wallet. Face recognition (MFA) is verified at both entry and exit gates before a transaction is created or completed.

Field Name	Data Type	Constraints	Description
Transaction_id	SERIAL / INT	PRIMARY KEY, NOT NULL	Unique identifier for each transaction.
User_id	INT	FOREIGN KEY (User.User_id), NOT NULL	The user who made this journey.
Source_station	INT	FOREIGN KEY (Station.Station_id), NOT NULL	The station where the user entered (scanned at entry gate via face recognition).
Destination_station	INT	FOREIGN KEY (Station.Station_id)	The station where the user exited (scanned at exit gate via face recognition).
Fare_amount	DECIMAL(10,2)	CHECK >= 0	The fare calculated based on DMRC slab and distance. Deducted from wallet on exit.

7.6 Role of Face Recognition as MFA

In traditional systems, users need to type a password or enter an OTP to log in or make a payment. In this system, face recognition works as Multi-Factor Authentication (MFA). Here is how it is used at each step:

Action	How MFA (Face Recognition) is Used
App Login	User opens the app. Camera scans face. If matched with Biometric_data, login is approved — no password needed.
Metro Gate Entry	User stands at the entry gate. Camera scans face. System checks face match AND wallet balance. If both pass, the gate opens and Source_station is recorded in Transaction table.
Metro Gate Exit	User stands at the exit gate. Camera scans face again to confirm identity. Destination_station is recorded and Fare_amount is deducted from Wallet.
Wallet Top-Up	Before adding money, face scan confirms the user's identity — no OTP needed.

8. Software & Hardware Requirements

8.1 Software Requirements

Software	Version / Specification	Purpose
Operating System	Windows 10/11 (64-bit) or Ubuntu 20.04 LTS	Development environment
Python	3.9+	Backend and AI processing
Java	JDK 11+	Route algorithm (Dijkstra's) from Smart Metro Navigator
FastAPI	Latest	REST API backend
Node.js	18+	Alternative/supplementary backend
React Native / Flutter	Latest stable	Mobile app
PostgreSQL	13 or above	Database
OpenCV	4.5+	Real-time face detection
Dlib	19.24+	Face landmark detection and embedding extraction
TensorFlow	2.x	CNN model training/inference
Git	Latest	Version control
VS Code	Latest	Primary code editor

8.2 Hardware Requirements

Component	Minimum Specification	Purpose
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5	Development machine
RAM	8 GB minimum	Running AI models and servers locally
Storage	256 GB SSD minimum	Project files and database
Camera	1080p HD Webcam	Face capture during development and testing
GPU (Optional)	NVIDIA with CUDA support	Faster AI model training
Internet	Stable broadband connection	API calls, cloud database (if used)

Edge Device	Raspberry Pi 4 / Jetson Nano	Gate-side face recognition hardware

9. Constraints & Assumptions

9.1 Constraints

- The system currently focuses on the Delhi Metro network only. Other cities' metro networks are not in scope.
- The face recognition system may perform with reduced accuracy if the user is wearing a full face mask, heavy makeup, or is in very poor lighting conditions.
- The gate hardware (cameras and edge devices) is assumed to be provided for testing purposes — the team will simulate this in software during development.
- The wallet is a closed-loop system; real-money payment gateway integration (like Razorpay or Paytm) is planned but depends on availability of sandbox API keys.
- User data privacy is governed by India's Digital Personal Data Protection Act, 2023.

9.2 Assumptions

- DMRC station data (coordinates, line maps, fare slabs) is publicly available and can be used for academic purposes.
- The team has access to the existing GitHub codebases: Smart Metro Navigator and MFAS.
- The face recognition model will be tested in a controlled lab environment with consented volunteer photos.
- All four team members have access to personal laptops meeting the minimum hardware requirements.

10. References

1. IEEE Standard 830-1998: Recommended Practice for Software Requirements Specifications.
2. Shashi Pandey. Smart Metro Navigator. GitHub, 2025. Available: github.com/Shashi0704/SMART_METRO_NAVIGATOR
3. Yash Agrawal. MFAS: Multi-Factor Authentication System Framework. GitHub, 2025. Available: github.com/IMYashAgrawal/MFAS
4. M. Malamut. 'Dijkstra's Pathfinder: Shortest Path Analysis in the Washington D.C. Metro.' Honors Thesis, Coastal Carolina University, 2021.
5. J. Doe et al. 'Optimal Routing in Urban Road Networks: A Graph-Based Approach Using Dijkstra's Algorithm.' MDPI Applied Sciences, vol. 15, no. 8, 2025.
6. M. Kotkar et al. 'Face Recognition Using ML in Public Transports.' IJTSRD, vol. 5, no. 6, 2021.
7. OpenCV Foundation. OpenCV Documentation: Face Recognition. 2025.
8. 'Blockchain-based vs. SQL Database Systems for Digital Twin Evidence Management.' arXiv, 2025.
9. R. Hayes and K. Rangarajan. 'MRTS: A Robust and Scalable Architecture for Metro Rail Ticketing System.' ResearchGate, 2015.
10. Delhi Metro Rail Corporation (DMRC). Official Network Map and Fare Metrics. 2026.

— End of Document —